

第7章

簡単な Web アプリケーションの作成①: 電卓

前章までの内容により、Go 言語で基本的なロジックを組むことができます。いったん、ここまで学習した知識を使って、電卓の Web アプリケーションを作成してみましょう。本章では、Web アプリケーションを作成する上で必要な Web サーバと HTML のフォームについて説明し、最後に HTML のフォームを使用した電卓を実装します。

しかし、このリテラル「0.3」を格納した変数にリテラル「0.4」を加算してみても、リテラル「0.7」を格納した変数の値とは一致しません。「0.3」の変数に「0.4」を加算した場合は「0.7000000476837158203125」、「0.7」を格納した変数は「0.69999998079071044921875」になります。この差は、「0.3」に「0.4」を加算したとき、指数部の桁合わせなどが発生したためです。

また、定数同士での「0.3 + 0.4」の場合とも異なるので注意してください。定数同士での計算は定数式として扱われ、コンパイル時に計算します。

```
package main

import (
    "fmt"
    "math"
)

func main() {
    var f1 float32 = 0.3
    var b1 uint32 = math.Float32bits(f1)
    fmt.Printf("%.24f\n", f1) // 0.300000011920928955078125
    fmt.Printf("%032b\n", b1) // 00111110100110011001100110100

    f1 += 0.4
    var f2 float32 = 0.7
    b1 = math.Float32bits(f1)
    var b2 uint32 = math.Float32bits(f2)

    fmt.Printf("%.24f\n", f1) // 0.700000047683715820312500
    fmt.Printf("%.24f\n", f2) // 0.69999998079071044921875
    fmt.Printf("%032b\n", b1) // 0011111100110011001100110100
    fmt.Printf("%032b\n", b2) // 0011111100110011001100110011
    fmt.Println(f1 == f2)    // false

    var f3 float32 = 0.3 + 0.4
    var b3 uint32 = math.Float32bits(f3)
    fmt.Printf("%.24f\n", f3) // 0.69999998079071044921875
    fmt.Printf("%032b\n", b3) // 0011111100110011001100110011
    fmt.Println(f1 == f3)    // false
}
```

7.1 Web サーバの起動と HTML のフォーム

7.1.1 Web サーバの起動

Go 言語には、HTTP のサーバとクライアントを提供する「net/http パッケージ」が用意されています。net/http パッケージを使用することで簡単に Web サーバを構築できます。ここでは、まず、net/http パッケージを使用した Web サーバのコードの書き方について説明します。ブラウザに「Hello」と表示する Web サーバのコードは、次のとおりです。

```
package main

import (
    "fmt"
    "net/http"
)

// ServeHTTPメソッド用の構造体型
type Server struct{}

// httpリクエストを受け取るメソッド
func (Server) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    // HTMLの文字列 ... ①
    h := `
<html><head><title>Hello</title></head><body>
Hello
</body></html>

    // クライアント (ブラウザ) にHTMLを送信 ... ②
    fmt.Fprint(w, h)
}

func main() {
    // Webサーバを起動 ... ③
    http.ListenAndServe(":4000", Server{})
}
```

このプログラムで http.ListenAndServe 関数を呼び出すと、関数の中で Web サーバとしての

処理を実行し続けます。ブラウザから「http://localhost:4000」にアクセスしてみてください。すると、図 7.1 のように「Hello」が表示されます。なお、Web サーバのプログラムを終了するには、「CTRL」キーを押しながら「C」キーを押します。

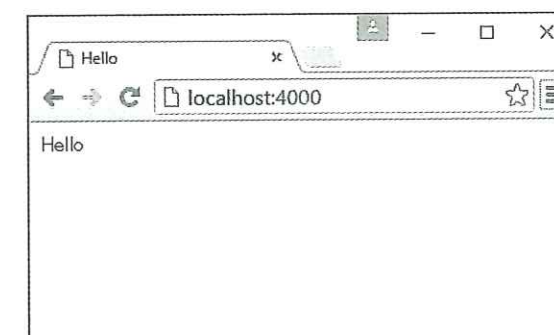


図 7.1 Hello の表示

上記のコードには、まだ紹介していない文法（構造体など）が含まれます。この章では、そのあたりの文法は気にせずに読み進めてください。電卓の Web アプリケーションを作成する上で重要な点は次の 3 つです。

① HTML の文字列

クライアントに送信する HTML を、文字列として変数 h に格納します。なお、HTML を記述する際は、raw 文字列リテラル（「`」（バッククォート）による文字列リテラル）を使用しています。

② クライアント (ブラウザ) に HTML を送信

fmt.Fprint 関数は、出力先を指定できる Print 関数です。1 番目の引数で出力先を指定し、2 番目以降の引数を出力します。ここでは、http.ResponseWriter 型の変数 w を出力先に指定することで、クライアント (ブラウザ) に html コードを送信することができます。

③ Web サーバを起動

http.ListenAndServe 関数を呼び出すと、Web サーバの処理が始まります。Web サーバのポート番号は、1 番目の引数で決まります。ここでは、ポート番号に「4000」を指定しています。

7.1.2 HTML のフォーム

次に、HTML のフォームによって文字列を送信する機能を追加します。HTML のフォームとは、テキストボックスやラジオボタンなどの入力項目を使用して、ブラウザからサーバにデータを送信する仕組みのことです。

ここで追加する機能は、フォームによるテキストボックスの文字列入力です。テキストボック

スに入力した文字列は、フォームの送信ボタン (submit タイプのボタン) を押すと、Web サーバに送信されます。Web サーバ側は、この文字列を HTML に組み込んで表示させます。

HTML のフォームを持つコードを以下に示します (以下のコードは、「7.1.1」で紹介したコードからの差分です)。ここでは、import 宣言と ServeHTTP メソッドの 2 つを変更してください。

```
import (
    "fmt"
    "html"      // htmlパッケージのインポートを追加 ... ①
    "net/http"
)

func (Server) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    // フォームの入力値を取得 ... ②
    v := r.FormValue("input_value")

    // HTMLの文字列 ... ③
    h := `
<html><head><title>HTMLのフォーム</title></head><body>
<form action="/" method="post">
    <input type="text" name="input_value">
    <input type="submit" name="送信"><br>
    入力値: ` + html.EscapeString(v) + `
</form>
</body></html>
`

    // クライアント (ブラウザ) にHTMLを送信
    fmt.Fprint(w, h)
}
```

このプログラムを実行してブラウザで「http://localhost:4000」を開くと、テキストボックスが表示されます。そして、このテキストボックスに文字列を入力して送信ボタンを押すと、図 7.2 のように入力した文字列が「入力値:」のところに出力されます。

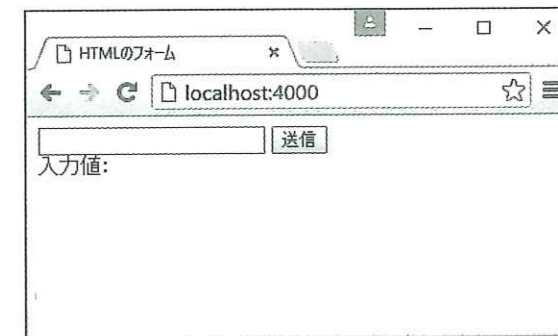


図 7.2 HTML のフォーム

「7.1.1」のコードから追加、変更した点は、次の 3 つです。

① html パッケージのインポートを追加

HTML エスケープ用の `html.EscapeString` 関数を使用するために、`html` パッケージをインポートします。HTML エスケープは、「<」「>」などを含む入力値を HTML タグとして出力しないようにするために用いられます。出力時に値を HTML エスケープすると、悪意のあるスクリプトを送り込むクロスサイトスクリプティング (XSS) を防ぐことができます。

② フォームの入力値を取得

`http.Request` の `FormValue` メソッドは、テキストボックスに入力された値を取得しています。このメソッドの引数は、テキストボックスとなる `input` タグの `name` 属性「`input_value`」を指定します。

③ HTML の文字列

フォームと入力値を出力する HTML を構築します。フォームの項目は、テキストボックスと送信ボタンの 2 つです。また、変数 `v` に格納した入力値は、HTML エスケープして出力します。

7.2 電卓の実装

7.2.1 電卓の Web アプリケーションの作成

最後に、本章の目的である電卓の Web アプリケーションを作成します。この電卓アプリは、「7.1 Web サーバの起動と HTML のフォーム」を拡張し、図 7.3 のように電卓の入力項目すなわち 2 つの数値と演算子の項目を追加したものです。そして、これらの項目の入力値を受け取ったサーバ側が、電卓としての演算結果を返します。

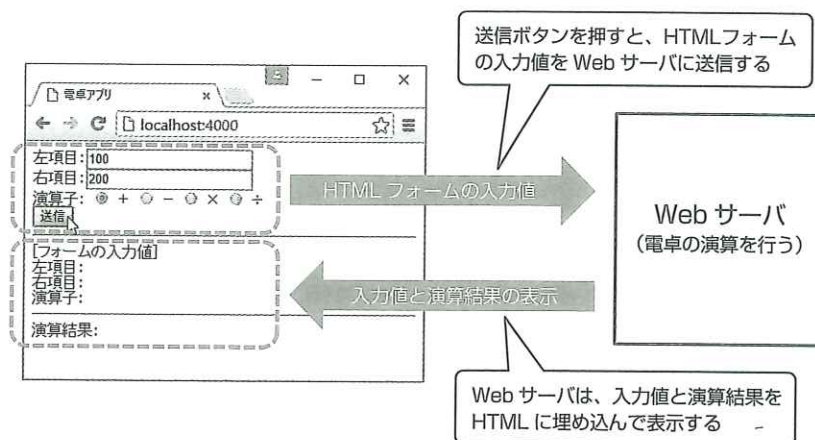


図 7.3 電卓アプリ

電卓アプリのコードを以下に示します (以下のコードは、「7.1.2」で紹介したコードからの差分です)。ここでは、import 宣言と ServeHTTP メソッドの 2 つを変更してください。

```
import (
    "fmt"
    "html"
    "net/http"
    "strconv" // strconv パッケージのインポートを追加 ... ①
)

func (Server) ServeHTTP(w http.ResponseWriter, r *http.Request) {
    // フォームの入力値を取得 ... ②
    left := r.FormValue("left") // 左項目 (テキストボックスの値)
    right := r.FormValue("right") // 右項目 (テキストボックスの値)
```

```
op := r.FormValue("op") // 演算子 (ラジオボタンの値)
```

```
// 文字列を整数に変換 ... ③
```

```
leftInt, leftErr := strconv.Atoi(left)
```

```
rightInt, rightErr := strconv.Atoi(right)
```

```
// 四則演算の処理 ... ④
```

```
// 変換エラーがなければ、演算子に従って計算
```

```
var result string
```

```
if (leftErr == nil) && (rightErr == nil) {
```

```
    // 演算子ごとに分岐
```

```
    switch op {
```

```
    case "add": // 加算
```

```
        result = strconv.Itoa(leftInt + rightInt)
```

```
    case "sub": // 減算
```

```
        result = strconv.Itoa(leftInt - rightInt)
```

```
    case "multi": // 乗算
```

```
        result = strconv.Itoa(leftInt * rightInt)
```

```
    case "div": // 除算
```

```
        result = strconv.Itoa(leftInt / rightInt)
```

```
    }
```

```
}
```

```
// HTML の文字列 ... ⑤
```

```
h := `
```

```
<html><head><title>電卓アプリ</title></head><body>
```

```
<form action="/" method="post">
```

```
    左項目: <input type="text" name="left"><br>
```

```
    右項目: <input type="text" name="right"><br>
```

```
    演算子:
```

```
    <input type="radio" name="op" value="add" checked> +
```

```
    <input type="radio" name="op" value="sub"> -
```

```
    <input type="radio" name="op" value="multi"> ×
```

```
    <input type="radio" name="op" value="div"> ÷
```

```
    <br><input type="submit" name="送信"><br>
```

```
[フォームの入力値]<br>
```

```
左項目: ` + html.EscapeString(left) + `<br>
```

```
右項目: ` + html.EscapeString(right) + `<br>
```

```
演算子: ` + html.EscapeString(op) + `<br>
```



```

    演算結果: ` + html.EscapeString(result) + `
</form>
</body></html>

// クライアント (ブラウザ) にHTMLを送信
fmt.Fprint(w, h)
}

```

上記のコードで表示される HTML のフォームは、数値を入力するテキストボックス「left」と「right」、演算子を選択するラジオボタン「op」です。これら3つの項目は電卓の入力値であり、たとえばテキストボックス「left」に左辺の値「100」、テキストボックス「right」に右辺の値「200」を入力し、ラジオボタン「op」で演算子「+」を選択します。入力した後に送信ボタンを押すと、入力値が Web サーバ側に送信されます。

Web サーバ側では、テキストボックスの値を演算対象とし、四則演算の種類をラジオボタンの値で分岐して決めます。そして演算した結果を、HTML に追加してブラウザ側に送信し、図 7.4 のように表示します。

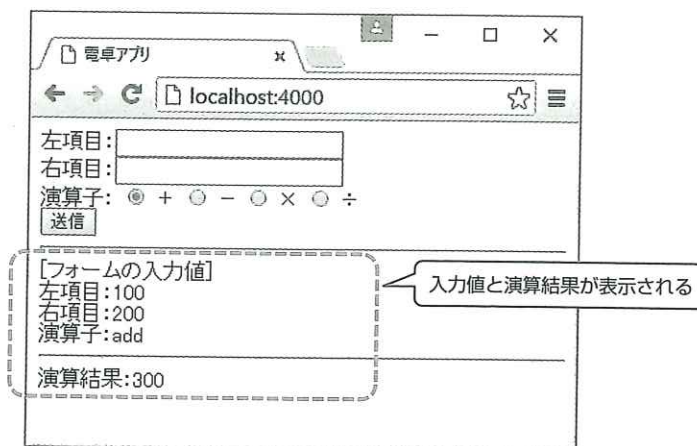


図 7.4 電卓アプリ (送信後)

「7.1.2」のコードから追加、変更した点は、次の5つです。

① strconv パッケージのインポートを追加

文字列を整数に変換する strconv.Atoi 関数と、整数を文字列に変換する strconv.Itoa 関数を使用するために、strconv パッケージをインポートします。

② フォームの入力値を取得

テキストボックスとラジオボタンに入力された値を取得します。取得する値は、すべて文字列型です。

③ 文字列を整数に変換

四則演算を行うため、strconv.Atoi 関数で入力値の文字列を整数に変換します。この関数の戻り値は、1つ目が整数、2つ目がエラー値です。エラー値が nil の場合のみ、正常に変換された整数を取得します。nil 以外の場合は、変換エラーとなります。

④ 四則演算の処理

エラーなしで文字列を整数に変換できた場合、入力値「op」に従って四則演算を行います。演算結果は、strconv.Itoa 関数で整数を文字列に変換して、変数 result に格納します。

⑤ HTML の文字列

フォーム、入力値、および演算結果を持つ HTML を構築します。

7.2.2 多倍長整数

ここまでで作成した電卓は、int 型で計算を行っていました。int 型に格納できる整数の桁数は、64 ビットの場合でも最大 19 桁です。より大きな数値を取り扱う場合は、多倍長整数を扱える math/big パッケージの big.Int 型を使用します。

big.Int 型は、多倍長整数を格納する型であり、通常の整数と同じように四則演算を行えます。電卓で入力した数値を big.Int 型の変数に設定すれば、より大きな数値も計算できるようになります。int 型から big.Int 型に変更したコードを以下に示します (「7.2.1」のコードからの差分です)。

```

import (
    "fmt"
    "html"
    "math/big" // math/big パッケージのインポートを追加 ... ①
    "net/http"
)

// 文字列を整数に変換 ... ②
leftInt := &big.Int{}
rightInt := &big.Int{}
_, leftOK := leftInt.SetString(left, 10)
_, rightOK := rightInt.SetString(right, 10)

// 四則演算の処理 ... ③

```



```
// 変換エラーがなければ、演算子に従って計算
var result string
if leftOK && rightOK {
    resultInt := &big.Int{}
    // 演算子ごとに分岐
    switch op {
    case "add": // 加算
        resultInt.Add(leftInt, rightInt)
    case "sub": // 減算
        resultInt.Sub(leftInt, rightInt)
    case "multi": // 乗算
        resultInt.Mul(leftInt, rightInt)
    case "div": // 除算
        resultInt.Div(leftInt, rightInt)
    }
    result = resultInt.String()
}
```

import 宣言と ServeHTTP メソッドの一部を変更しています。変更した点の詳細は、次のとおりです。

① `math/big` パッケージのインポートを追加

math/big パッケージを追加しています。もともとあった strconv パッケージは使わなくなるため、削除してください。

② 文字列を整数に変換

big.Int 型の変数を作成した後、SetString メソッドで文字列を big.Int 型に変換しています。SetString メソッドの第 1 引数は変換する文字列、第 2 引数は基数を渡します。戻り値は真偽値型であり、変換が成功したときに「true」、失敗したときに「false」を返します。

③ 四則演算の処理

演算子の代わりにメソッドを使用して、四則演算を行います。たとえば `big.Int` 型で足し算を行う `Add` メソッド「`resultInt.Add(resultLeft, resultRight)`」は、`int` 型の「`resultInt = resultLeft + resultRight`」と同じです。演算した結果は、`String` メソッドで文字列に変換します。

多倍長整数を使用することで、図 7.5 のように非常に大きな数値でも計算できるようになります。



図 7.5 多倍長整数を使用した電卓アプリ

COLUMN HTTP/2 対応

Go 言語のバージョン 1.6 から、HTTP/2 が net/http で使用できるようになりました。HTTP/2 は、従来の HTTP/1.1 から主に性能面の改良を目的として作成されたプロトコルです。HTTP/2 の特徴は次のとおりです。

- リクエスト／レスポンスの多重化

HTTP/1.1 では、1 つの TCP 接続で送信できるリクエストは 1 つのみで、次のリクエストを送信するにはレスポンスを待つか TCP 接続の数を増やすしかありませんでした。しかし HTTP/2 では、1 つの TCP 接続の中で複数のリクエストの送信が可能になり、次のリクエストを送信するのにレスポンスを待つ必要がなくなりました。

● 優先制御

多くの Web サイトは、HTML、CSS、JavaScript、および画像ファイルで構築されています。これらのうち HTML などのレスポンスを受信できれば、画像ファイルがなくても画面の描画を始められます。HTTP/2 ではこのような優先制御を行うため、HTML や画像ファイルなどをリクエストした際にどれを優先するかを決める「**重み**」と「**依存関係**」を指定できるようになっています。

- バイナリプロトコル

HTTP/1.1 はテキストプロトコルでした。一方、HTTP/2 はバイナリプロトコルを採用しています。バイナリプロトコルを採用したことで、転送に必要な容量が小さくなります。また、テキストプロトコルでは存在した空白文字の処理の違いなどを少なくできます。

●ヘッダ圧縮

「HPACK」と呼ばれる圧縮フォーマットによって、ヘッダ部分を圧縮します。

Go 言語のサーバで HTTP/2 を使用するには、ListenAndServeTLS 関数などで HTTPS を接続可能にしてください。この状態のサーバに HTTPS で接続すれば、自動的に HTTP/2 が有効になります。